🅿 Smart Parking Availability System

IoT-Based Final Project — Spring 2026

Garvin Patel
patelg5@mymail.nku.edu
Northern Kentucky University

The Problem

> *"Students waste time every day searching for parking with zero visibility into what's available."*

- 🚗 Drivers circle multiple garage levels searching for open spots
- ❌ No system to check availability **before entering**
- ⏰ Results in **wasted time**, traffic congestion, frustration
- 📚 Students arrive **late to class** because of parking

This is a real, daily problem at NKU that needed a real IoT solution.

The Solution

A fully working IoT system that **physically detects cars** and shows live availability on any device

```
HC-SR04 Ultrasonic Sensor
        ↓  detects distance every 3 seconds
XIAO ESP32C6 (WiFi)
        ↓  sends JSON via HTTP POST
Python Flask REST API
        ↓  stores to cloud
MongoDB Atlas Database
        ↓  React fetches every 3 seconds
Live React Dashboard
        ↓
Any device — phone, laptop, tablet
```

System Architecture

Layer	Technology	Purpose
Detection	HC-SR04 + ESP32C6	Measure distance, detect car
Protection	Logic Level Converter	5V → 3.3V signal conversion
Transmission	WiFi + HTTP POST	Send sensor data wirelessly
Backend	Python Flask	REST API receives + processes
Database	MongoDB Atlas	Cloud storage + history
Frontend	React.js	Live animated dashboard

Hardware Setup

- **HC-SR04 Ultrasonic Sensor** — sends sound pulse, measures distance to detect cars
- **XIAO ESP32C6** — microcontroller with built-in WiFi 6, reads sensor data
- **Logic Level Converter** — critical protection: converts 5V ECHO signal to safe 3.3V
- **Breadboard + Jumper Wires** — connects all components without soldering

Why Logic Level Converter Was Critical:

```
HC-SR04 ECHO outputs 5V
ESP32C6 GPIO accepts maximum 3.3V
Connecting directly = permanent hardware damage ⚠️
Solution: Level converter on ECHO line only ✅
```

How Detection Works

```
ESP32 sends 10-microsecond pulse on TRIG pin
                ↓
HC-SR04 fires ultrasonic sound wave
                ↓
Sound bounces off object and returns
                ↓
ECHO pin stays HIGH for travel duration
                ↓
distance = duration × 0.034 / 2  (cm)
                ↓
distance < 50cm  →  🔴 OCCUPIED
distance > 50cm  →  🟢 AVAILABLE
                ↓
Data sent to Flask API every 3 seconds
```

Sprint 1 — Hardware + Local Detection

Completed Milestones:

- ✅ Wired HC-SR04 to Arduino Nano — first sensor test
- ✅ Installed CH340 USB driver on Mac
- ✅ Live distance readings confirmed in Serial Monitor
- ✅ Identified critical 5V vs 3.3V voltage issue
- ✅ Added logic level converter to protect ESP32C6
- ✅ Rewired HC-SR04 directly to XIAO ESP32C6
- ✅ ESP32C6 connected to WiFi hotspot
- ✅ Live dashboard hosted directly from ESP32
- ✅ Dashboard shows OCCUPIED / AVAILABLE in real time

Sprint 2 — Backend + Dashboard

Completed Milestones:

- ✅ Python Flask REST API receiving sensor data via HTTP POST
- ✅ MongoDB Atlas cloud database storing spot readings + history
- ✅ Full data pipeline confirmed: ESP32 → Flask → MongoDB
- ✅ React.js dashboard with 120 animated parking spots
- ✅ SVG cars drawn in occupied spots, green circles for available
- ✅ Live stats: total spots, available count, occupancy rate %
- ✅ Filter buttons: All spots / Available only / Occupied only
- ✅ Hover tooltips: spot ID, status, distance in cm
- ✅ Auto-refreshes every 3 seconds from MongoDB via Flask API

Flask Backend Code

```python
@app.route('/update', methods=['POST'])
def update_spot():
    data = request.get_json()
    spot_id = data.get('spot_id', 'A1')
    distance = data.get('distance_cm', 0)
    status = "OCCUPIED" if distance < 50 else "AVAILABLE"

    spots_collection.update_one(
        {"spot_id": spot_id},
        {"$set": {
            "status": status,
            "distance_cm": distance,
            "last_updated": datetime.now().strftime("%H:%M:%S")
        }},
        upsert=True
    )
    return jsonify({"status": status})

@app.route('/status', methods=['GET'])
def get_status():
    spots = list(spots_collection.find({}, {"_id": 0}))
    occupied = sum(1 for s in spots if s["status"] == "OCCUPIED")
    return jsonify({"spots": spots, "occupied": occupied,
                    "available": len(spots) - occupied})
```

MongoDB Data Structure

spots collection — current status:

```
{
  "spot_id": "A1",
  "status": "OCCUPIED",
  "distance_cm": 12.3,
  "last_updated": "21:15:50"
}
```

readings collection — full history:

```
{
  "spot_id": "A1",
  "distance_cm": 12.3,
  "status": "OCCUPIED",
  "timestamp": "2026-04-19T21:15:50"
}
```

React Dashboard

```
useEffect(() => {
  const fetchData = async () => {
    const res = await fetch("http://192.168.50.68:5001/status");
    const data = await res.json();
    setSpots(data.spots);
    setStats({ total: data.total,
               available: data.available,
               occupied: data.occupied });
  };
  fetchData();
  const interval = setInterval(fetchData, 3000);
  return () => clearInterval(interval);
}, []);
```

- **120 spots** across 6 rows (A–F, 20 spots each)

Hardest Problems Solved With AI

Problem	Root Cause	Solution
5V damages ESP32	ECHO is OUTPUT pin	Logic level converter
CH340 driver error	Mac doesn't support natively	Downloaded + installed manually
Bootloader error	Wrong ATmega setting	ATmega328P Old Bootloader
WiFi status code 6	Wrong password	Simplified hotspot password
Flask response -1	Different network subnets	Same WiFi for all devices
MongoDB SSL error	Python 3.14 certificate issue	tlsAllowInvalidCertificates=True
Chrome HTTPS redirect	Browser auto-upgrades HTTP	Typed http:// explicitly
Port 5000 blocked	Mac AirPlay uses port 5000	Switched to port 5001

Learning With AI — Topic 1

Embedded C++ Sensor Programming

Key concepts learned:

- `pulseIn()` — measures microsecond-level echo duration
- `pinMode()` — every GPIO pin must be INPUT or OUTPUT
- `digitalWrite()` — sends HIGH/LOW pulse to TRIG pin
- Distance formula: `duration × 0.034 / 2`
- ECHO is OUTPUT — sensor sends voltage TO ESP32
- Serial Monitor at 115200 baud for hardware debugging
- ATmega328P Old Bootloader for Arduino Nano uploads

Learning With AI — Topic 2

IoT Networking — ESP32 to Cloud Pipeline

Key concepts learned:

- `WiFi.begin()` — connects ESP32, gets assigned IP address
- IP subnets — devices must be on same range to communicate
- `HTTPClient` — sends POST requests with JSON payload
- Flask `@app.route` — defines API endpoints
- `pymongo` + `upsert=True` — update without duplicating
- React `useEffect` + `setInterval` — live polling every 3s
- CORS — allows React on port 3000 to fetch Flask on port 5001
- HTTP vs HTTPS — ESP32 only supports unencrypted HTTP

What I Discovered

Things I learned that weren't in any lecture:

- **Voltage matters physically** — wrong voltage permanently destroys hardware
- **Networks are strict** — one wrong IP address and nothing communicates
- **Error codes tell the story** — WiFi status 6, HTTP -1, SSL errors all have specific meanings
- **AI debugging is powerful** — paste an error, get the exact fix with explanation
- **Hardware + software must match** — baud rates, bootloaders, protocols all need to align

> *"Every error I encountered taught me something I could not have learned from a textbook."*

Progress Summary

Week	Milestone
Week 1-3	Project planning, PPP, problem definition
Week 4-5	Hardware setup, first sensor test, CH340 driver
Week 6-7	Voltage fix, ESP32C6 integration, S1P submitted
Week 8-9	Flask API built, MongoDB Atlas connected
Week 10-11	React dashboard with 120 animated spots
Week 12-13	Full pipeline testing, dashboard polished
Week 14-15	Final presentation, GitHub updated, rubric submitted

Live Demo 🎬

What you will see:

1. HC-SR04 sensor detecting distance in real time
2. ESP32C6 sending data to Flask API every 3 seconds
3. MongoDB Atlas storing every reading in the cloud
4. React dashboard updating live — OCCUPIED / AVAILABLE
5. Put hand in front of sensor → dashboard changes in 3 seconds

Full pipeline: Sensor → WiFi → Flask → MongoDB → React

Thank You 🎉

Garvin Patel | patelg5@mymail.nku.edu
Northern Kentucky University | Spring 2026

> *"From zero electronics knowledge to a full-stack IoT system — built with real hardware, real data, and real AI assistance."*

GitHub: [your repo link]
Demo Video: [your video link]